

Algorithm for file updates in Python

Project description

I am a security professional working at a healthcare company. As part of my job, I am required to regularly update a file that identifies the employees who can access restricted patient records via specific IP addresses. To ensure this restricted subnetwork remains secure, I developed a Python algorithm to automate the process. This algorithm checks a text file containing an allow list against a separate remove list, revokes access for any matching IP addresses, and rewrites the updated list back to the file.

Open the file that contains the allow list

```
import_file = "allow_list.txt"
with open(import_file, "r") as file:
```

First, I assigned the name of the text file containing the allowed IP addresses ("allow_list.txt") to the variable `import_file`. Then, I used a `with` statement alongside the `open()` function to open the file in read mode ("r"). The `with` statement is best practice here because it ensures that the file automatically closes once the block of code finishes executing, preventing resource leaks. Inside the `with` statement, the opened file is temporarily assigned to the variable `file`.

Read the file contents

```
ip_addresses = file.read()
```

Inside the `with` statement, I used the `.read()` method to read the entire contents of the file. This method converts the file's text contents into a single large string. I stored this string in a variable called `ip_addresses` so I could manipulate it in the following steps.

Convert the string into a list

```
ip_addresses = ip_addresses.split()
```

In order to remove individual IP addresses from the allow list, the data needs to be in a list format rather than one continuous string. I used the `.split()` method on the `ip_addresses`

string. By default, `.split()` separates the string by whitespace, converting it into a list of individual IP address strings. I then reassigned this new list back to the `ip_addresses` variable.

Iterate through the remove list

```
for element in remove_list:
```

I needed to process a second list called `remove_list`, which contains the IP addresses that must be revoked. To do this, I set up a `for` loop to iterate through each IP address in the `remove_list`. I used `element` as the loop variable to represent each individual IP address as the loop evaluates them one by one.

Remove IP addresses that are on the remove list

```
if element in ip_addresses:  
    ip_addresses.remove(element)
```

Inside the `for` loop, I created a conditional `if` statement to evaluate whether the current loop variable (`element`) exists in the `ip_addresses` list. If the condition evaluates to `True`, I apply the `.remove()` method to the `ip_addresses` list, passing in the `element` to delete it.

Applying the `.remove()` method in this way is possible and highly effective because there are no duplicate IP addresses in the `ip_addresses` list, meaning it successfully drops the targeted IP on the first pass without leaving lingering unauthorized access.

Update the file with the revised list of IP addresses

```
ip_addresses = "\n".join(ip_addresses)  
with open(import_file, "w") as file:  
    file.write(ip_addresses)
```

Now that the unauthorized IP addresses were removed from the variable, I needed to update the original file. First, I converted the `ip_addresses` list back into a string using the `.join()` method. I applied `.join()` to the newline character ("`\n`") so that each IP address is placed on a new line, making the text file easy to read. Finally, I used another `with` statement and the `open()` function, this time with the `"w"` (write) parameter, to open the file. I used the

`.write()` method to write over the file's old contents with the newly secured `ip_addresses` string.

Summary

I created a Python algorithm that automates the process of updating an access control list for restricted patient data. The script uses the `with open()` statement to securely open the `allow_list.txt` file and reads its contents as a string, then converts it into a list using `.split()` for easy manipulation. Next, it uses a `for` loop to iterate through a list of IP addresses slated for removal, utilizing an `if` statement to check if they exist in the allow list and applying the `.remove()` method to delete them if they do. Finally, the script uses the `.join()` method to convert the updated list back into a readable string and overwrites the original text file using the `.write()` method. This automation ensures that restricted subnetwork access remains secure, accurate, and up-to-date.